

مستندات پروژه docker-anywhere

کامل فارسی – عامل اجرای دستور از راه دور با ارتباط خروجی محور (Outbound-Only) برای مدیریت Docker و سرور تولید راهنمای

خروجی) و Control Server مرکزی (صف دستورات و تاریخچه). هیچ پورت ورودی برای اجرای دستور از اینترنت روی سرور تولید باز نیست. خلاصه: این پروژه دو بخش دارد – Agent روی سرور تولید (فقط درخواست

۱. معرفی کلی پروژه

Python، UV ساخته شده است. هدف: مدیریت امن Docker و سرور از راه دور بدون باز کردن API ورودی روی تولید. پروژه docker-anywhere با FastAPI

مشکل API ورودی: هر مهاجم می‌تواند به POST /execute حمله کند.

راه‌حل: Agent هر ۱۵ ثانیه Control Server را poll می‌کند، فقط دستورات Whitelist اجرا می‌شود، نتیجه برمی‌گردد.

دو پوشه اصلی

اجرا، گزارش نتیجه، poll: روی سرور تولید – agent/

سرور مرکزی: صف دستور، احراز هویت، تاریخچه – control_server/

۲. معماری و جریان کار

```
Operator --POST /api/v1/commands--> Control Server
Agent <--GET /api/v1/poll----- Control Server (every 15s)
Agent --POST /api/v1/results-----> Control Server
Agent: only GET /health on 127.0.0.1 (local monitoring)
```

۱. اپراتور دستور را با کلید Operator به صف می‌فرستد.

۲. بعدی دستور را می‌گیرد poll در Agent.

۳. executor.py (بدون argv دستور را به shell=True) تبدیل می‌کند.

۴. نتیجه stdout/stderr/returncode به Control Server برمی‌گردد.

۳. پوشه agent / – عامل روی سرور تولید

FastAPI نقطه ورود – agent/main.py

هیچ GET /: systemmd وضعیت عامل برای GET /health: poller توقف shutdown: poller تنظیم لاگ و شروع startup: اجرای دستور از راه دور وجود ندارد endpoint پاسخ ساده.

Pydantic تنظیمات – agent/config.py

مسیرهای LOG_FILE و Docker، ALLOWED_SCRIPTS_DIR، REBOOT_CONFIRMATION_TOKEN، timeout، agent/.env: AGENT_ID، API_KEY، HMAC_SECRET، CONTROL_SERVER_URL، POLL_INTERVAL_SECONDS خواندن

مدل‌های داده – agent/models.py

ActionType (enum مجاز)، CommandPayload، ExecutionResult، HealthResponse، CommandStatus.

امنیت – agent/security.py

هدرها: HMAC-SHA256 با signed_request و signed_headers. setup_logging و audit_event (لاگ JSON). X-API-Key، X-Timestamp، X-Nonce، X-Signature. RateLimiter برای /health.

اجرای دستور – agent/executor.py

فقط از: regex ثابت. اعتبارسنجی نام سرویس با argv به action هر map _build_command. confirmation_token نیاز به: server_reboot. path traversal ضد، ALLOWED_SCRIPTS_DIR. نقطه ورود اصلی: execute_command. timeout و asyncio.

polling حلقه – agent/poller.py

AgentPoller: httpx AsyncClient، GET poll، execute، POST results. backoff نگرهاری در خطا. last_poll_at نمایی در خطا. health برای commands_executed.

agent/.env.example

مستندات docker-anywhere
نمونه متغیرهای محیط – کپی به env.

agent/agent.service

نمونه hardening امنیتی با systemd

agent/__init__.py

علامت‌گذاری پکیج Python

۴. پوشه `control_server/` – سرور کنترل

سرور مرکزی. داده‌ها فعلاً in-memory است – برای تولید واقعی PostgreSQL/Redis پیشنهاد می‌شود.

`control_server/main.py` – API

سلامت سرور – GET /health

Operator اپراتور: صف کردن دستور (کلید – POST /api/v1/commands)

pending برداشتن دستور: Agent – GET /api/v1/poll?agent_id=...

ارسال نتیجه اجرا: Agent – POST /api/v1/results

اپراتور: مشاهده تاریخچه – GET /api/v1/history

`control_server/config.py`

AGENT_API_KEY/HMAC (مشترک با Agent)، OPERATOR_API_KEY/HMAC، MAX_TIMESTAMP_SKEW، rate limit.

`control_server/security.py`

verify_signed_request: API key، timestamp، nonce، HMAC. ضد replay و timing attack.

`control_server/models.py`

agent.models. استفاده مجدد از – PushCommandRequest، QueuedCommand، HistoryEntry

`control_server/cli.py`

دستور امضا شده بدون نوشتن کد push اپراتور برای CLI

`control_server/.env.example`

نمونه تنظیمات سرور کنترل

۵. فایل‌های ریشه

(fastapi، httpx) و وابستگی‌ها UV تعریف پروژه – pyproject.toml

agent/main.py (ریشه) – نسخه قدیمی آزمایشی؛ جایگزین main.py

Python (۳.۱۲+) نسخه – python-version

۶. دستورات مجاز (Whitelist)

- service ری‌استارت سرویس (نیاز به `docker_restart`)
- `docker_rebuild` — `build` و `up -d`
- `docker_logs` — لاگ (پیش‌فرض ۱۰۰ `params.tail`)
- `docker_status` — `docker compose ps`
- توقف / شروع — `docker_stop` / `docker_start`
- `server_reboot` — ری‌بوت (نیاز به `confirmation_token`)
- `run_script` — اسکریپت از پوشه مجاز (`params.script_name`)

۷. امنیت

- فقط ارتباط خروجی از Agent — بدون پورت دستور ورودی
- فقط `/health 127.0.0.1`
- روی هر درخواست `API key + HMAC`
- `replay` ضد `timestamp` و `Nonce`
- ثابت `argv` با `subprocess` — بدون `shell` — `Whitelist`
- کلیدها: `chmod 600` — `hex 32` — `openssl rand` روی `env`.
- در تولید `Control Server` اجباری برای `HTTPS`

۸. نحوه اجرا با UV

```
uv sync
cp control_server/.env.example control_server/.env
uv run uvicorn control_server.main:app --host 0.0.0.0 --port 8000
```

```
cp agent/.env.example agent/.env
uv run uvicorn agent.main:app --host 127.0.0.1 --port 8080
```

```
export OPERATOR_API_KEY=...
export OPERATOR_HMAC_SECRET=...
uv run python -m control_server.cli push \
  --agent-id prod-server-01 --action docker_restart --service web
```

۹. افزودن دستور جدید

۱. مقدار جدید به `ActionType` در `agent/models.py`
۲. شاخه در `build_command()` داخل `agent/executor.py` — فقط `argv` ثابت
۳. ارسال از `CLI` یا `API` — هرگز رشته `shell` خام قبول نکنید

هشدار: هر action جدید باید ورودی validate شود و به دستور از پیش تعریف شده map شود.

۱۰. مسیر کامل: از عضو جدید تا توسعه‌دهنده حرفه‌ای

جمله بنویسید که «الان چه می‌دانم». هدف: شناخت کامل ساختار، خواندن، کاوش، و توسعه امن این پروژه در ۳ تا ۵ هفته. این بخش را عنوان به عنوان نخوانید. مراحل شماره‌دار را به ترتیب انجام دهید. بعد از هر مرحله یک

مرحله ۰ – ساختار کامل مخزن (قبل از هر کدی)

```
docker-anywhere/
pyproject.toml, .python-version, uv.lock
agent/ -> main, config, models, security, executor, poller, .env.example, agent.service
control_server/ -> main, config, models, security, cli, .env.example
docs/ -> documentation-fa.html, generate_pdf.py, fonts/
```

دو پروسه جدا: Control Server (پورت 8000، اپراتور + عامل) و Agent (پورت 8080 محلی، فقط /health).

مرحله ۱ – راه‌اندازی و اولین اجرا (روز ۱-۲)

گام ۱.۱: نصب Python 3.12+ و UV. اجرای uv sync در ریشه پروژه.

env. عامل = AGENT_API_KEY = سرور. AGENT_ID=prod-server-01. HMAC_SECRET = AGENT_HMAC_SECRET.

گام ۱.۲: کپی agent/.env.example به env. و control_server/.env.example به API_KEY.

گام ۱.۳: ترمینال ۱ – uv run uvicorn control_server.main:app --host 0.0.0.0 --port 8000. باز کردن /health و /docs. باید ۵ endpoint ببینید.

گام ۱.۴: ترمینال ۲ – uv run uvicorn agent.main:app. ظرف ۱۵ ثانیه poll در لاگ دیده شود. /health باید poller_running=true نشان دهد.

گام ۱.۵: ترمینال ۳ – push

دستور docker_status با command_id. CLI. بگیرید. در لاگ عامل اجرا را ببینید. commands_executed افزایش یابد.

گام ۱.۶: رسم دیاگرام از حافظه: اپراتور - POST commands - صف - poll - executor - results - history

مرحله ۲ – خواندن هر فایل به ترتیب وابستگی (روز ۳-۷)

گام ۲.۱: pyproject.toml و UV، pydantic-settings، httpx، FastAPI، python-version.

گام ۲.۲: CommandStatus، ActionType (۸ action)، agent/models.py. تمرین: توکن reboot در params است.

گام ۲.۳: QueuedCommand، HistoryEntry، PushCommandRequest، control_server/models.py. ببینید ActionType از agent.models import می‌شود.

گام ۲.۴: agent/config.py و agent/.env.example + config.py و control_server/.env.example + جدول تطابق env بین دو سرویس بنویسید.

گام ۲.۵: signed_headers (agent/security.py) و verify_signed_request (control_server/security.py) – رشته کانیکال، compare_digest، timestamp، nonce. سپس

گام ۲.۶: _queues – control_server/main.py. هر حافظه. history_ در حافظه. GET poll، POST results، GET history، POST commands، route: POST commands. نقش Operator vs Agent. control_server/main.py

گام ۲.۷: control_server/cli.py – تابع sign_ و push_command و argparse.

گام ۲.۸: _build_command – agent/executor.py. برای هر _run_subprocess، action، بدون shell، run_script، server_reboot با path traversal check، execute_command، confirmation_token.

گام ۲.۹: agent/poller.py – حلقه backoff، httpx، poll، به‌روزرسانی state برای health.

گام ۲.۱۰: agent/main.py – lifespan، agent/executor.py، توقف poller، فقط health/ و ./.

گام ۲.۱۱: ExecStart – agent/agent.service و systemd hardening.

مرحله ۳ – کاوش عملی هر مسیر (هفته ۲)

گام ۳.۱: هر Docker action را با CLI تست کنید: rebuild، start، stop، restart، logs، status. لاگ عامل و history را بخوانید.

گام ۳.۲: تست رد امنیتی – کلید اشتباه (۴۰۱)، reboot بدون (rejected token)، نام سرویس نامعتبر (validation).

گام ۳.۳: breakpoint در poller.py، main.py (push)، دریافت دستور)، executor.py (execute_command). یک دستور را قدم‌به‌قدم debug کنید.

گام ۳.۴: git log --oneline -20 – ببینید تیم اخیراً روی چه چیزی کار کرده.

مرحله ۴ – اولین کارهای توسعه (هفته ۳)

– (docker_pull در models.py، شاخه در build_command()) با argv ثابت، تست با CLI، به روزرسانی مستندات.

گام ۴.۱: افزودن action جدید (مثلا ActionType

گام ۴.۲: تغییر POLL_INTERVAL_SECONDS در agent/.env – مشاهده poll سریع تر در /health.

گام ۴.۳: افزودن لاگ ساختاریافته در executor با audit_event موجود.

بدون دستور خام از شبکه، secret فقط در auth، env.example جدا برای operator و agent، اعتبارسنجی Pydantic.

گام ۴.۴: چک لیست PR – بدون shell=True،

مرحله ۵ – تولید و عملیات (هفته ۴)

پشت (Nginx + TLS HTTPS). کلید با 32 hex - rand openssl. برنامه جایگزینی صف in-memory با PostgreSQL/Redis.

گام ۵.۱: Control Server

، uv sync با 600 chmod، کاربر در گروه 127.0.0.1:8080/health curl، docker، systemd enable secure-agent.

گام ۵.۲: نصب agent روی لینوکس – agent/.env

– عامل poll نمی کند (لاگ، شبکه، کلید)، دستور pending ماند (agent_id یا agent خاموش)، (stderr failed در history).

گام ۵.۳: عیب یابی

مرحله ۶ – تسلط حرفه ای (هفته ۵ به بعد)

نگاه به کد توضیح دهید: تفاوت auth اپراتور و عامل، صف چند دستور، چرا outbound polling، خطر restart سرور کنترل.

گام ۶.۱: بدون

گام ۶.۲: بتوانید اضافه کنید: API، action اپراتور، persistence، config، تست.

چک لیست تسلط (همه باید برقرار باشد)

- رسم معماری و نقش هر فایل بدون یادداشت

- خواندن خط به خط همه فایل های Python در agent و control_server

- اجرای هر ۸ ActionType (یا دلیل عدم اجرا)

- در سه لایه debugger در trace -

- حداقل یک تغییر action (end-to-end، لاگ، یا config)

- توضیح HMAC و ضد replay به همکار

- استقرار agent با systemd و control server با HTTPS

- شناخت شکاف تولید (in-memory، نبود تست) و پیشنهاد راه حل

کند، و هر PR که shell خام از شبکه می پذیرد را رد کند. نسخه انگلیسی کامل با جزئیات بیشتر در documentation-fa.html بخش ۱۰. توسعه دهنده حرفه ای روی این پروژه کسی است که بدون نقشه فایل درست را باز کند، باگ را از history تا executor در چند دقیقه trace